UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

CS 498: Mixed Precision Training

Fan Lai

The Grainger College of Engineering

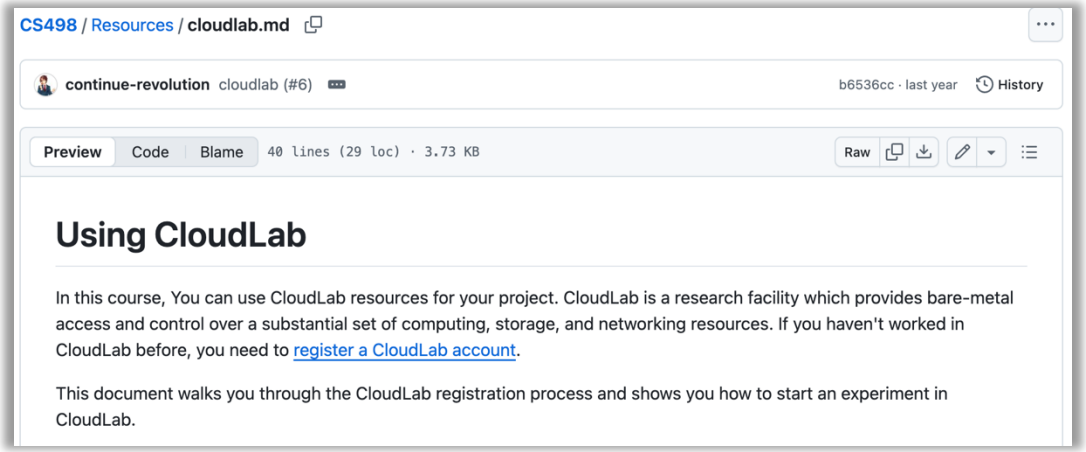
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and many colleagues

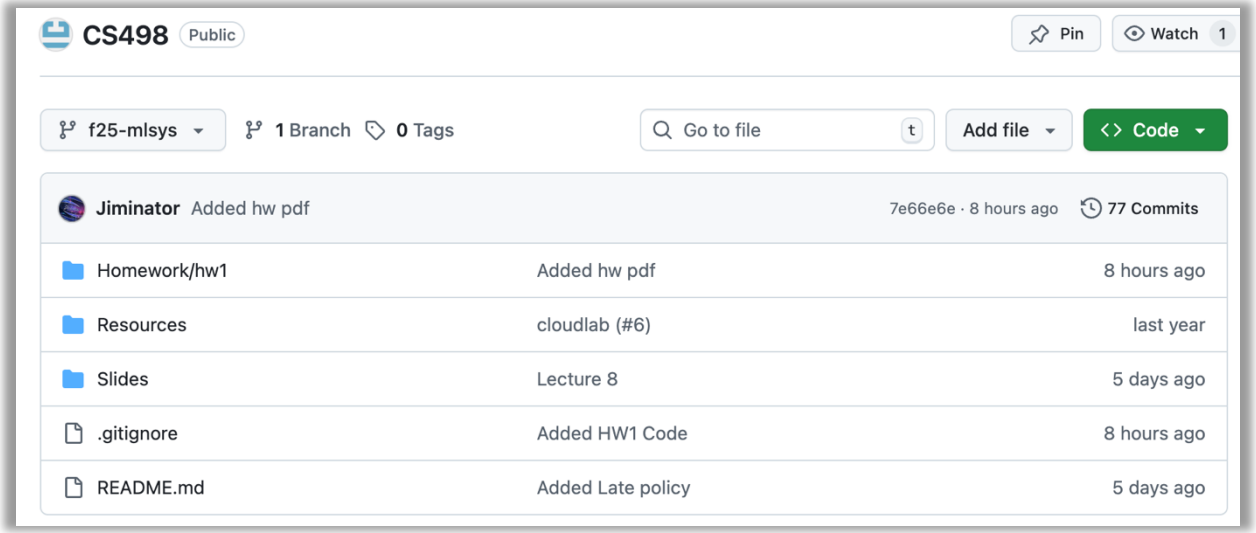
Assignment 1 is Released (Due on 10/15)

Five short questions (10 pts)

Lab: Parameter-server based DP (10 pts)

Lab: All-reduce implementation (30 pts)





We will use CPUs on CloudLab.

Cluster Assignments:

- HW1_Cluster_1: Alaybeyi-Hong
- HW1_Cluster_2: Hu-Lu
- HW1_Cluster_3: Luo-Sankaran
- HW1_Cluster_4: Sen-Wu
- HW1_Cluster_5: Xie-Zhuang

Assignment 1 is Released (Due on 10/15)

Five short questions (10 pts)

Lab: Parameter-server based DP (10 pts)

Lab: All-reduce implementation (30 pts)

NO Class this Friday! Meeting to discuss project ideas (SC 3128)

Project proposal due on 10/01!

Recall: Combining Model Parallelism

Model Parallelism:

- Tensor parallelism: communication-heavy
- Pipeline parallelism: tuning microbatch sizes to minimize bubbles
- Data parallelism: cannot run if layer (model) size exceeds GPU capacity

Minimizing compute and memory requirements is the key

Today

- **Mixed Precision Hardware**
- **What is Mixed Precision Training?**
- **Considerations for Mixed Precision**
- **Mixed Precision Software**

2006 – Now: Compute and Scaling

Compute scaling

2006 – Now: Compute and Scaling

Compute scaling

Tensor Cores: Matrix Multiplication Units

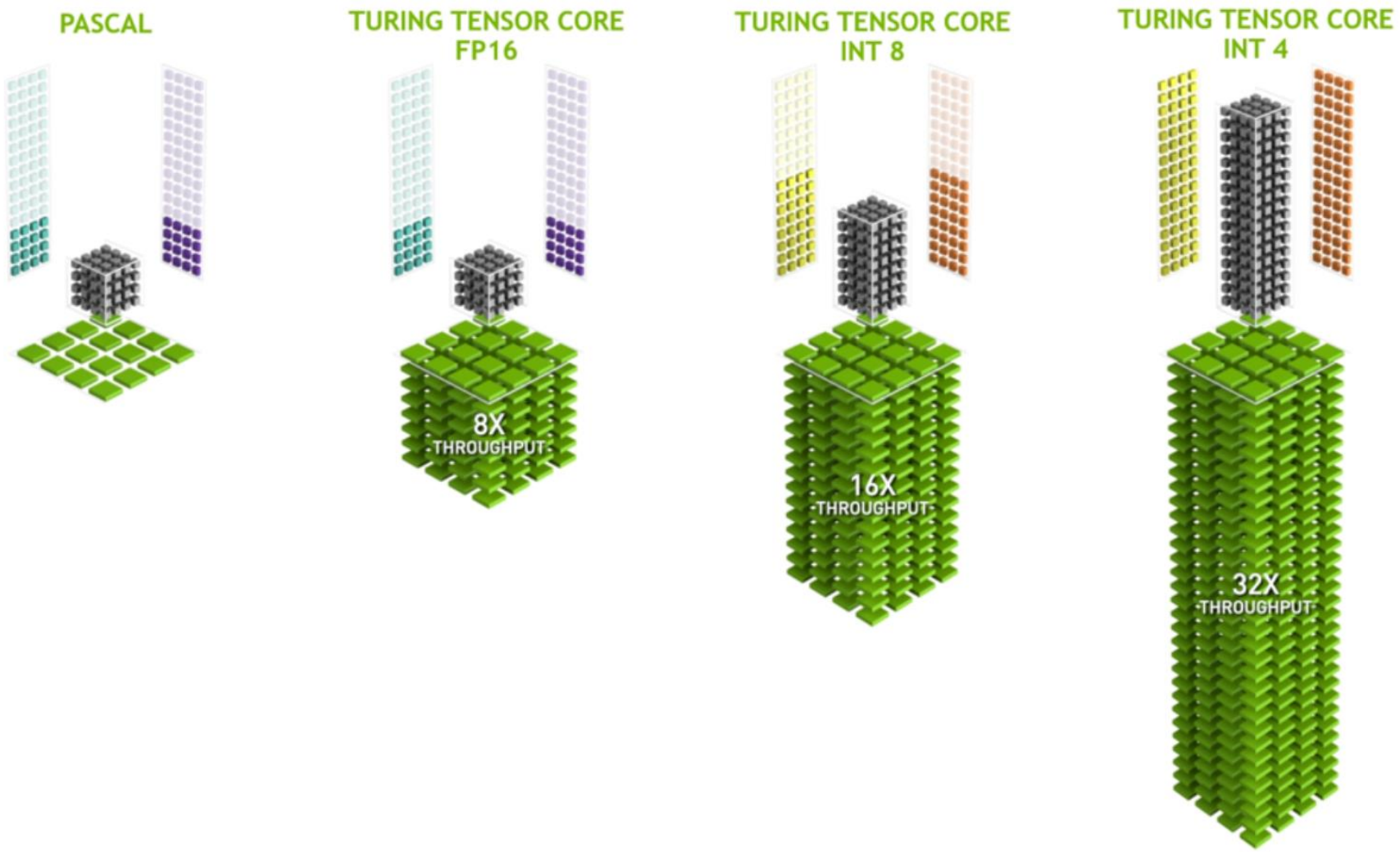
- **Tensor cores are:**
  - Special hardware execution units
  - Execute matrix multiply operations in parallel
  - Fused multiply-add on small matrices in one instruction cycle
  - Built to accelerate deep learning

Tensor Cores: Matrix Multiplication Units

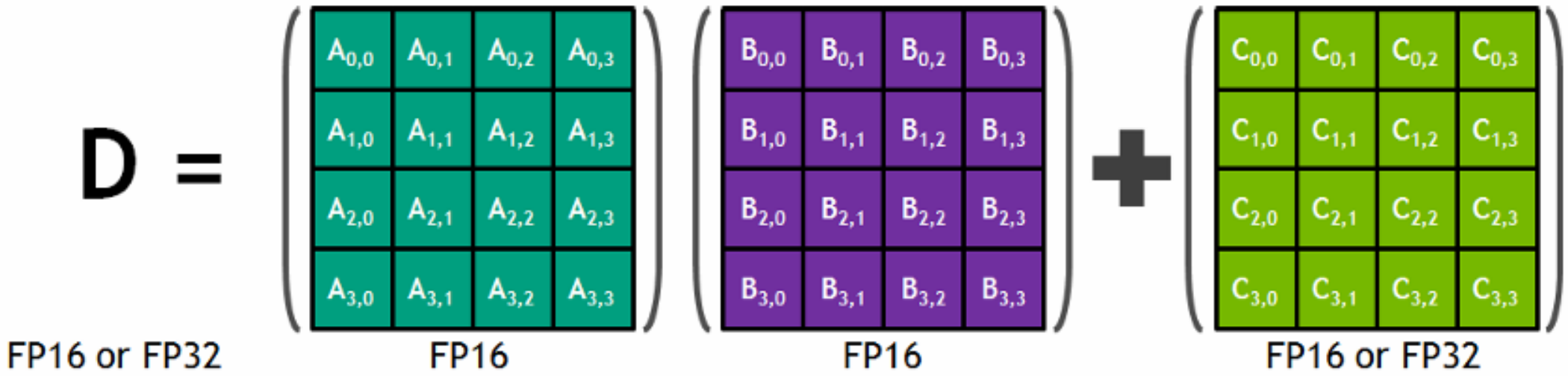
- **Tensor cores are:**
  - Special hardware execution units
  - Execute matrix multiply operations in parallel
  - Fused multiply-add on small matrices in one instruction cycle
  - Built to accelerate deep learning

- **Different flavors**
  - (V100) Volta Tensor Cores FP16
  - (T4) Turing Tensor Cores FP16/INT8/INT4
  - …

Tensor Cores: Different Flavors



Tensor Cores: Mixed Precision Matrix Math on 4x4 Matrices



$$D = AB + C$$

What is Mixed Precision Training?

What is Mixed Precision Training? In a Nutshell

- Idea that you can train deep neural networks in multiple precisions:
  - Make precision decisions per layer or operation
  - Full precision (Fp32) where needed to *maintain task-specific accuracy*
  - Reduced precision (Fp16) everywhere else for speed and scale

What is Mixed Precision Training? In a Nutshell

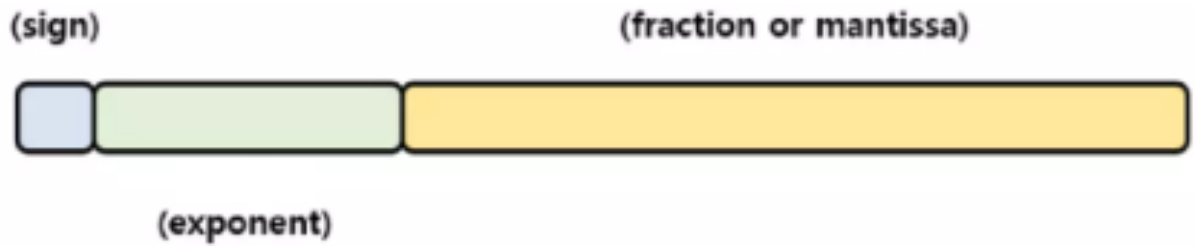
- Idea that you can train deep neural networks in multiple precisions:
  - Make precision decisions per layer or operation
  - Full precision (Fp32) where needed to *maintain task-specific accuracy*
  - Reduced precision (Fp16) everywhere else for speed and scale

- By using *multiple* precisions, we can have the best of both worlds: **speed** and **accuracy**

- Goal: accelerate deep neural network training with mixed precision under the constraints of **matching accuracy of full precision training** and **no changes to how model is trained**

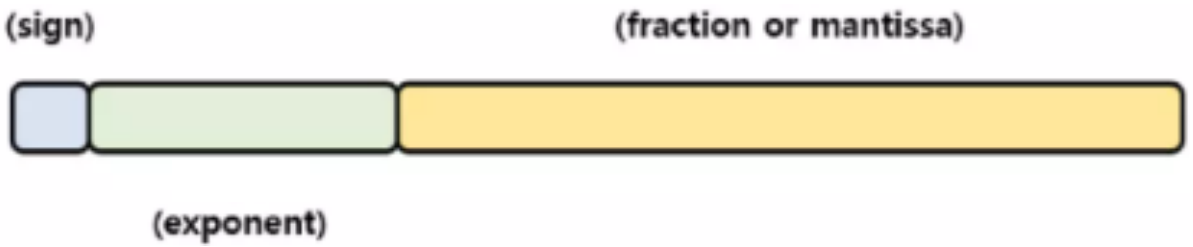
IEEE 754 Floating Point Representation Recap

- Number (e.g., 2.45) can be represented by $(-1)^S * (1.M) * 2^{(E - Bias)}$



IEEE 754 Floating Point Representation Recap

- Number (e.g., 2.45) can be represented by $(-1)^S * (1.M) * 2^{(E - Bias)}$



Half Precision (Floating Point 16)	1 bit	5 bit	10 bit
Single Precision (Floating Point 32)	1 bit	8 bit	23 bit
Double Precision (Floating Point 64)	1 bit	11 bit	52 bit
Quadruple Precision (Floating Point 128)	1 bit	15 bit	113 bit

Mixed Precision Training Benefits

- **Accelerates speed**
  - Tensor cores are 8x faster than FP32

Mixed Precision Training Benefits

- **Accelerates speed**
  - Tensor cores are 8x faster than FP32
- **Reduces memory bandwidth and footprint pressure**
  - FP16 halves memory traffic compared to FP32
  - FP16 halves the size of activation and gradient tensors

Why Not Use Pure FP16?

- Models cannot always converge properly in pure FP16
  - FP16 has a narrower dynamic range than FP32
  - May cause underflow/overflow issues and other arithmetic issues

Why Not Use Pure FP16?

- Models cannot always converge properly in pure FP16
  - FP16 has a narrower dynamic range than FP32
  - May cause underflow/overflow issues and other arithmetic issues

- **Weight updates**
  - Optimizer takes very **small increments** when search narrows into a solution
  - Late updates often cannot be represented in FP16, but can be crucial for accuracy

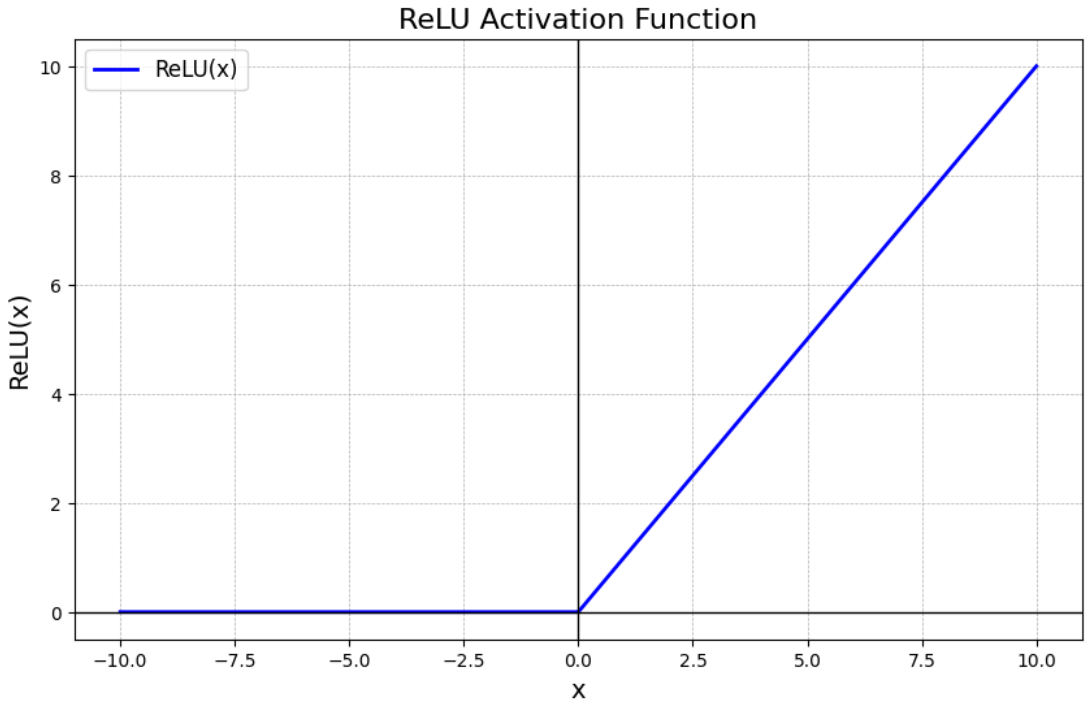
Considerations for Mixed Precision

Considerations for Mixed Precision Training

- Goal #1: Make mixed precision training general purpose, not only for limited class of applications
- Goal #2: With no changes to hyperparameters or the model architecture

Precision of OPS: Conservative Recommendations

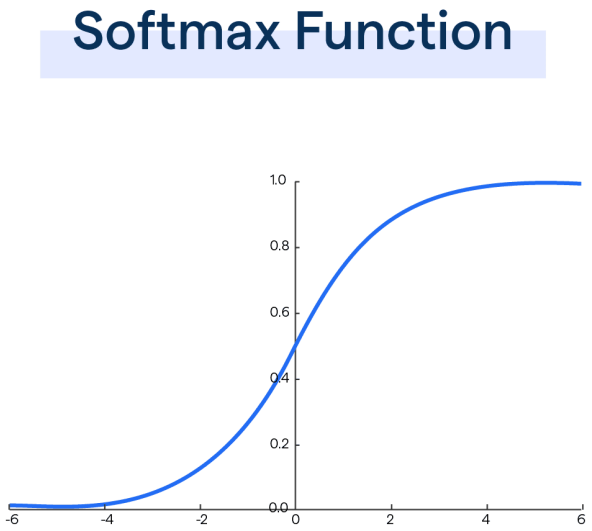
- Operations that can use FP16
  - Matrix multiplications
  - Most element-wise operations (e.g., relu, tanh, add, sub)



Precision of OPS: Conservative Recommendations

- Operations that can use FP16
  - Matrix multiplications
  - Most element-wise operations (e.g., relu, tanh, add, sub)

- Operations that need FP32 mantissa
  - Reduction operations (e.g., softmax, layernorm)



Precision of OPS: Conservative Recommendations

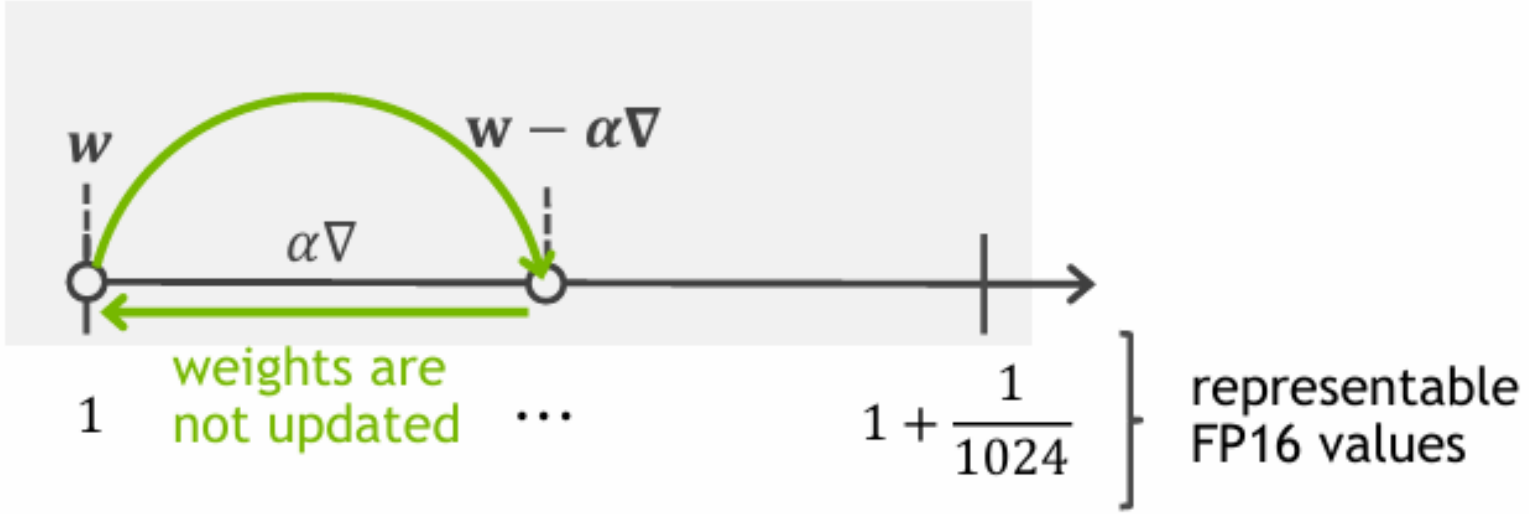
- Operations that can use FP16
  - Matrix multiplications
  - Most element-wise operations (e.g., relu, tanh, add, sub)
- Operations that need FP32 mantissa
  - Reduction operations (e.g., softmax, layernorm)
- Operations that need FP32 range
  - Element-wise operations where |f(x)| >> |x|, e.g., exp, log, pow
  - Loss functions

Master Weights

- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha\nabla$

Master Weights

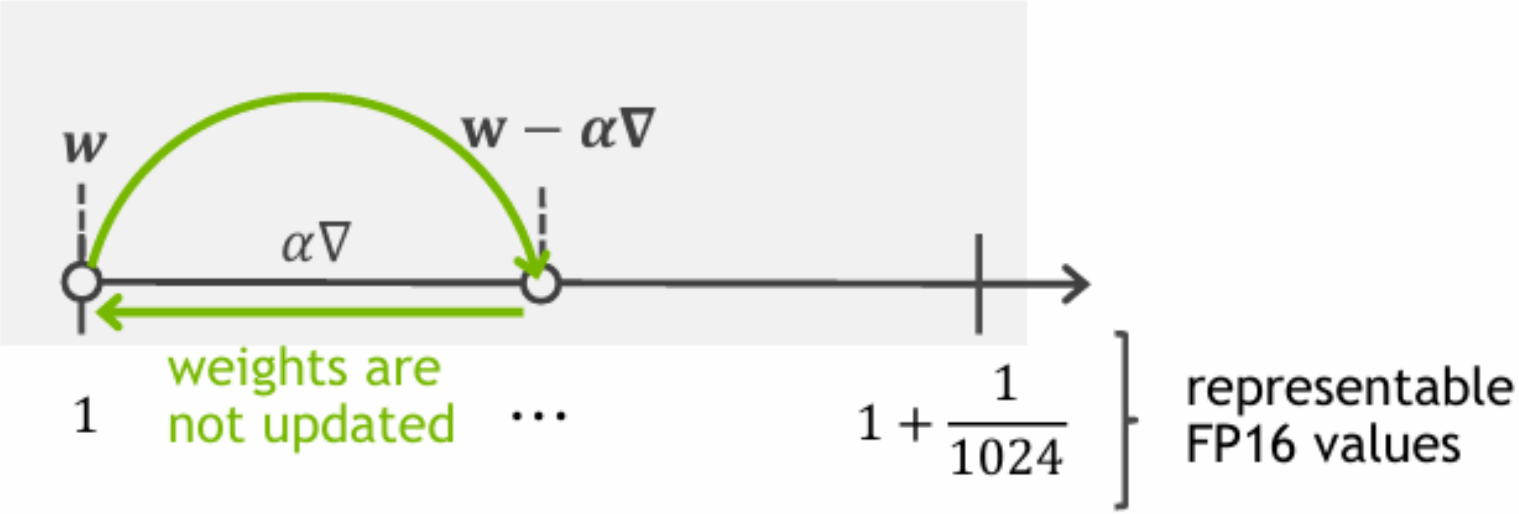
- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha\nabla$



Example:
$w \times 0.01$ leads to $\sim 2^7$ ratio,
$w \times 0.001$ leads to $\sim 2^{10}$ ratio

Master Weights

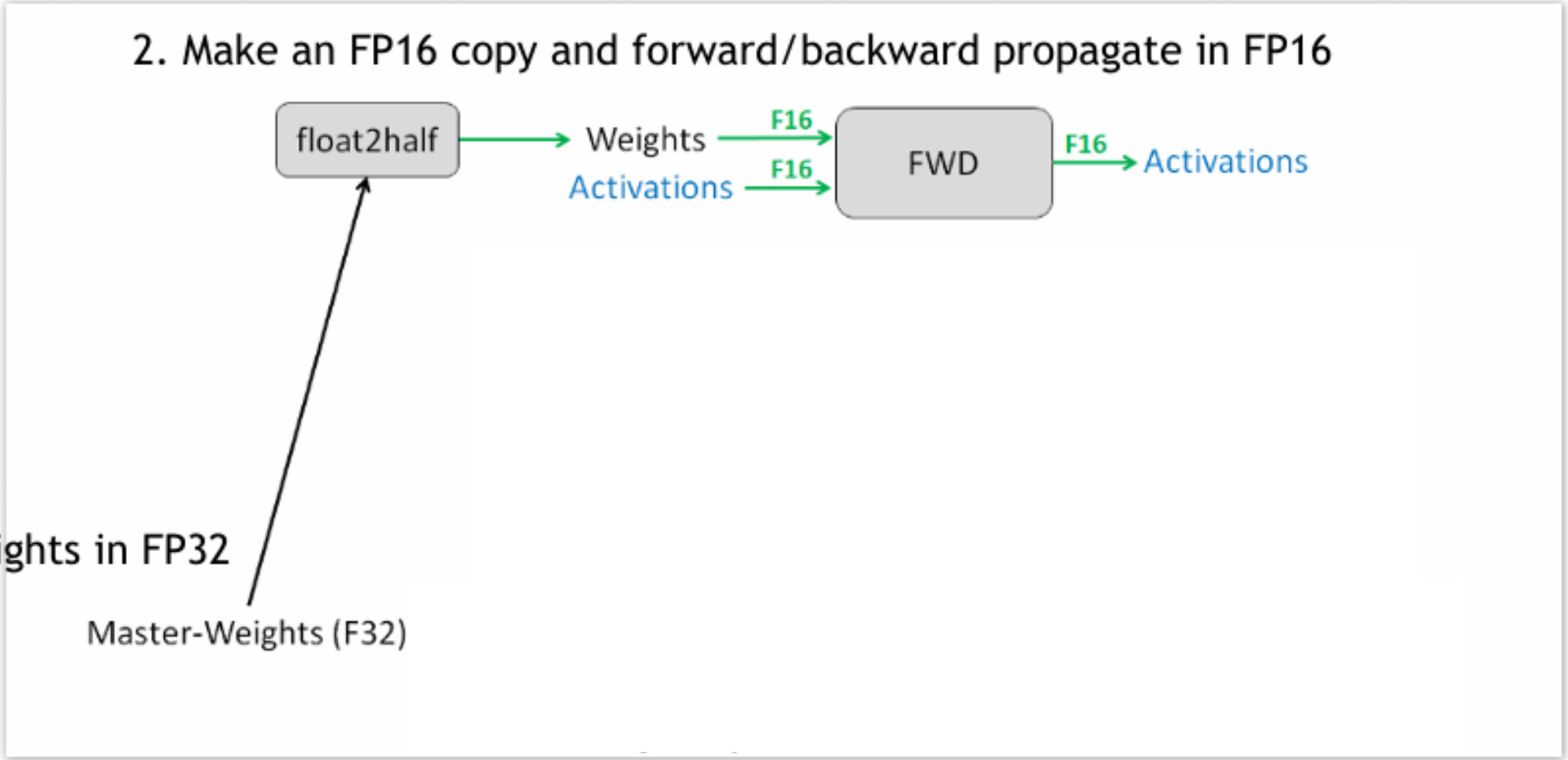
- **Problem:** in late stages of training, weight updates become too small for addition in FP16
- **Consequence:** weight update gets clipped to zero when $w \gg \alpha\nabla$



Example:
$w \times 0.01$ leads to $\sim 2^7$ ratio,
$w \times 0.001$ leads to $\sim 2^{10}$ ratio

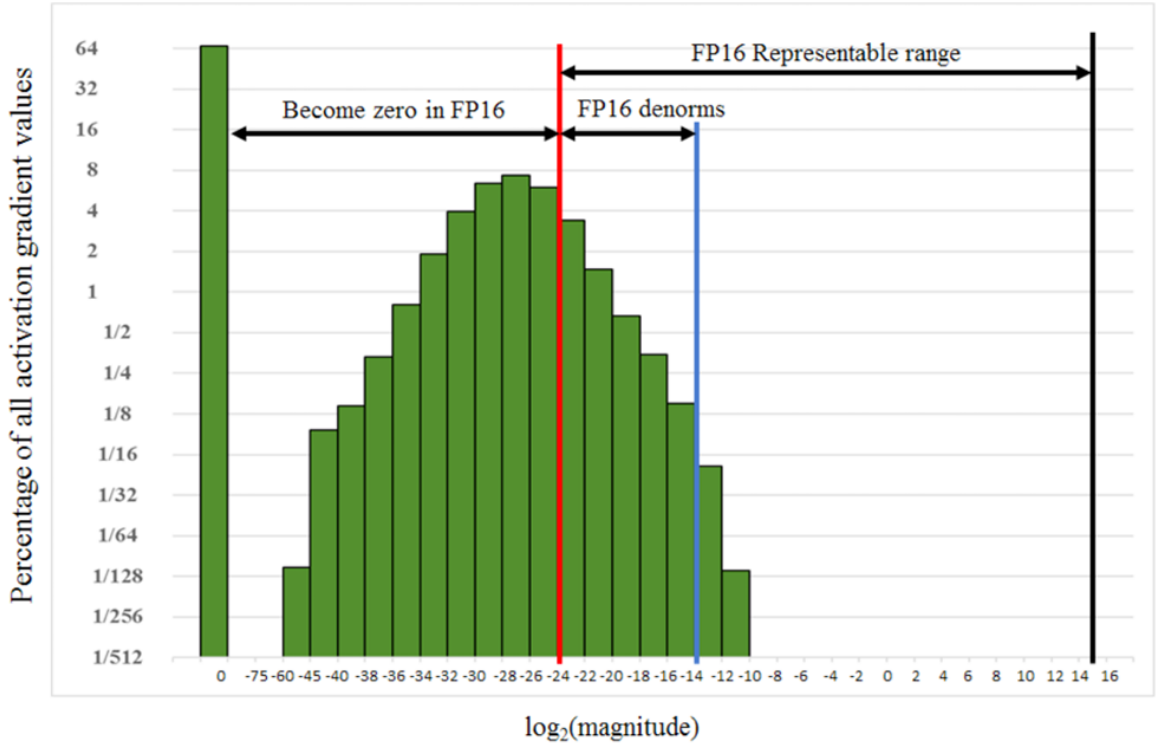
- **Conservative solution:** keep *master copy* of weights and *optimizer state* in FP32 so small updates can accumulate

Master Weights: Illustration



Loss Scaling: Put All Tensors in FP16 Range

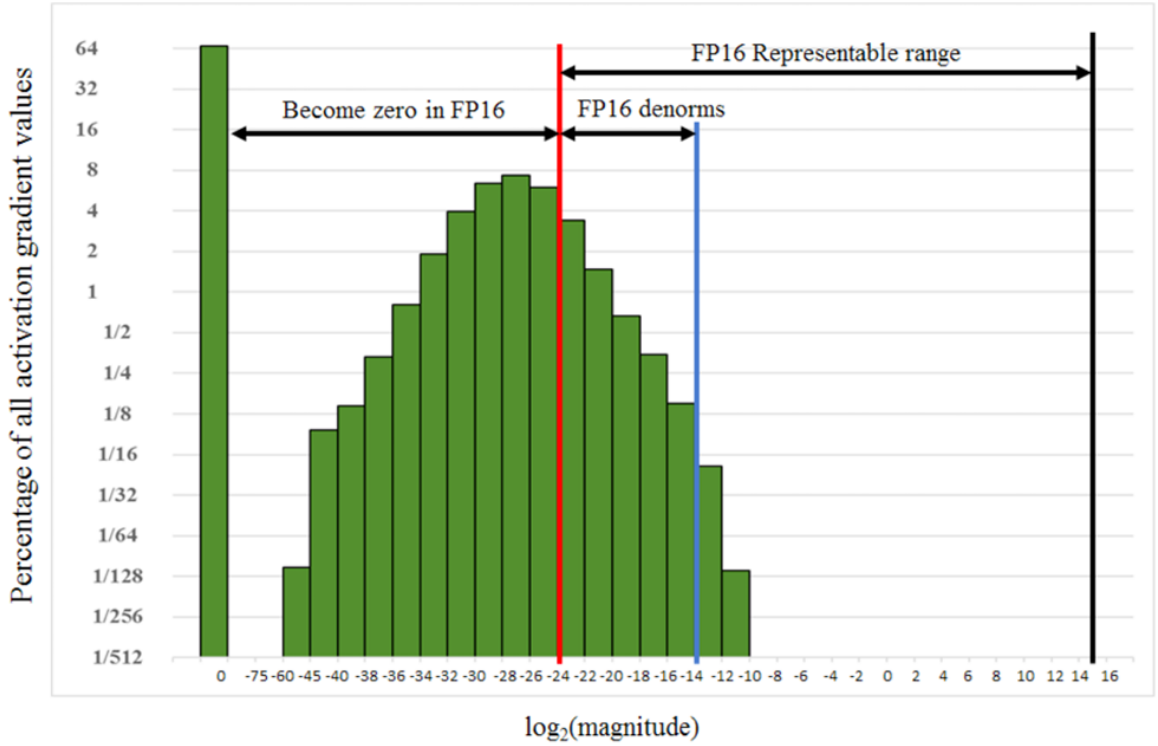
- Range representable in FP16: ~40 powers of 2



Histogram of activation gradient values

Loss Scaling: Put All Tensors in FP16 Range

- Range representable in FP16: ~40 powers of 2
- Gradients are small:
  - If cast to Fp16, some lost to zero
  - Much of fp16 representable ranges was left unused

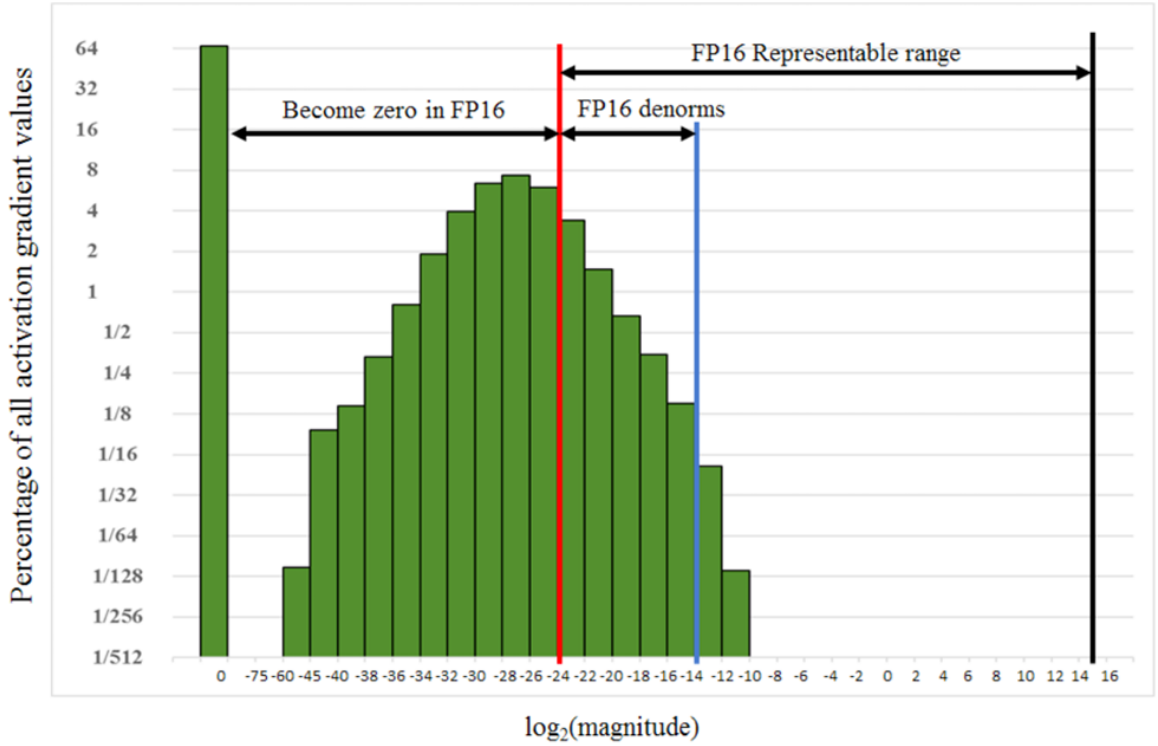


Histogram of activation gradient values

Loss Scaling: Put All Tensors in FP16 Range

- Range representable in FP16: ~40 powers of 2
- Gradients are small:
  - If cast to Fp16, some lost to zero
  - Much of fp16 representable ranges was left unused

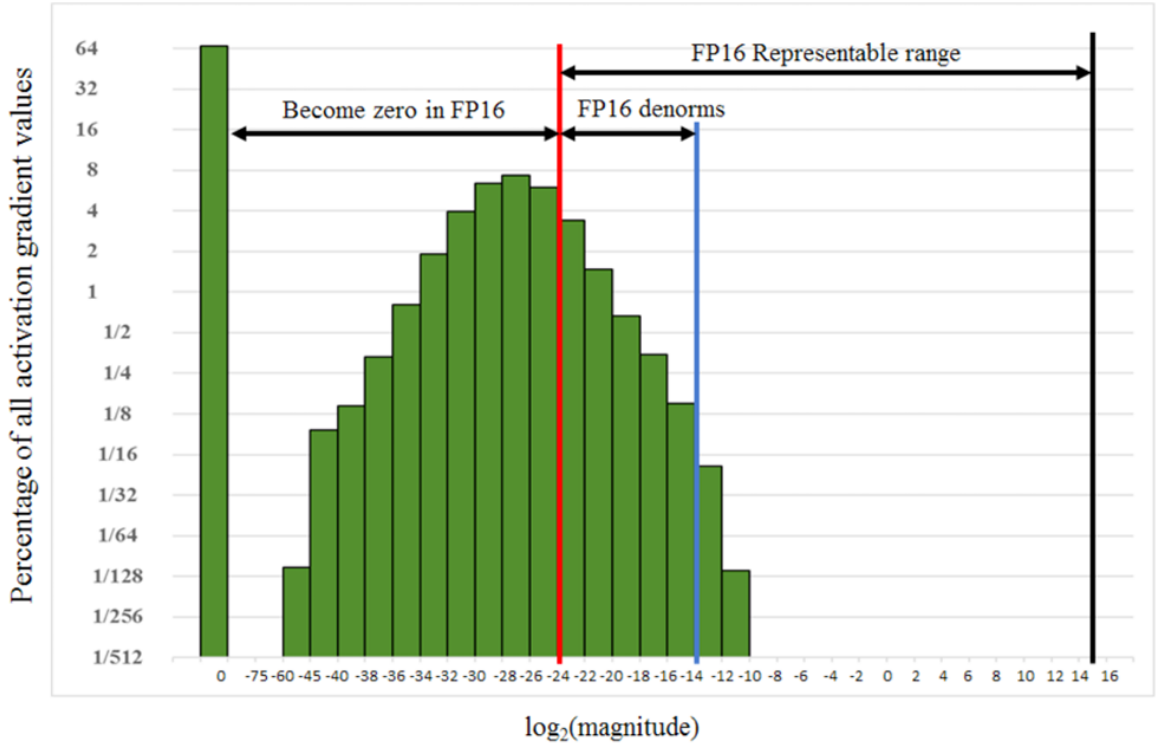
Question: How can we make better use of FP16 representable range to capture gradients?



Histogram of activation gradient values

Loss Scaling: Put All Tensors in FP16 Range

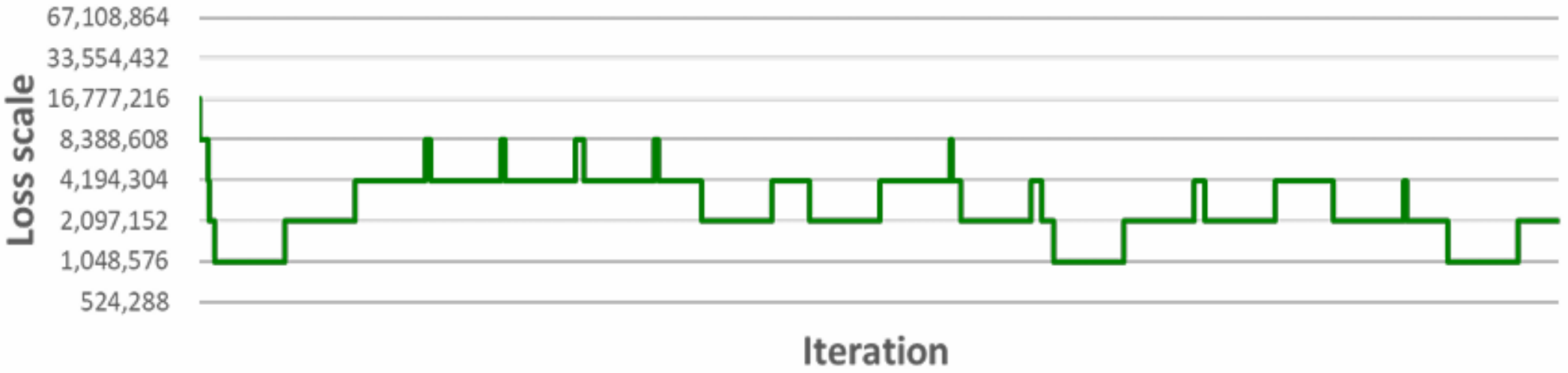
- Range representable in FP16: ~40 powers of 2
- Gradients are small:
  - If cast to Fp16, some lost to zero
  - Much of fp16 representable ranges was left unused
- Solution:
  - **Shift** small gradients values to fp16 range
  - Scaling gradients during backpropagation to prevent underflow
  - Gradients are scaled before backpropagation begins and rescaled before updating weights



Histogram of activation gradient values

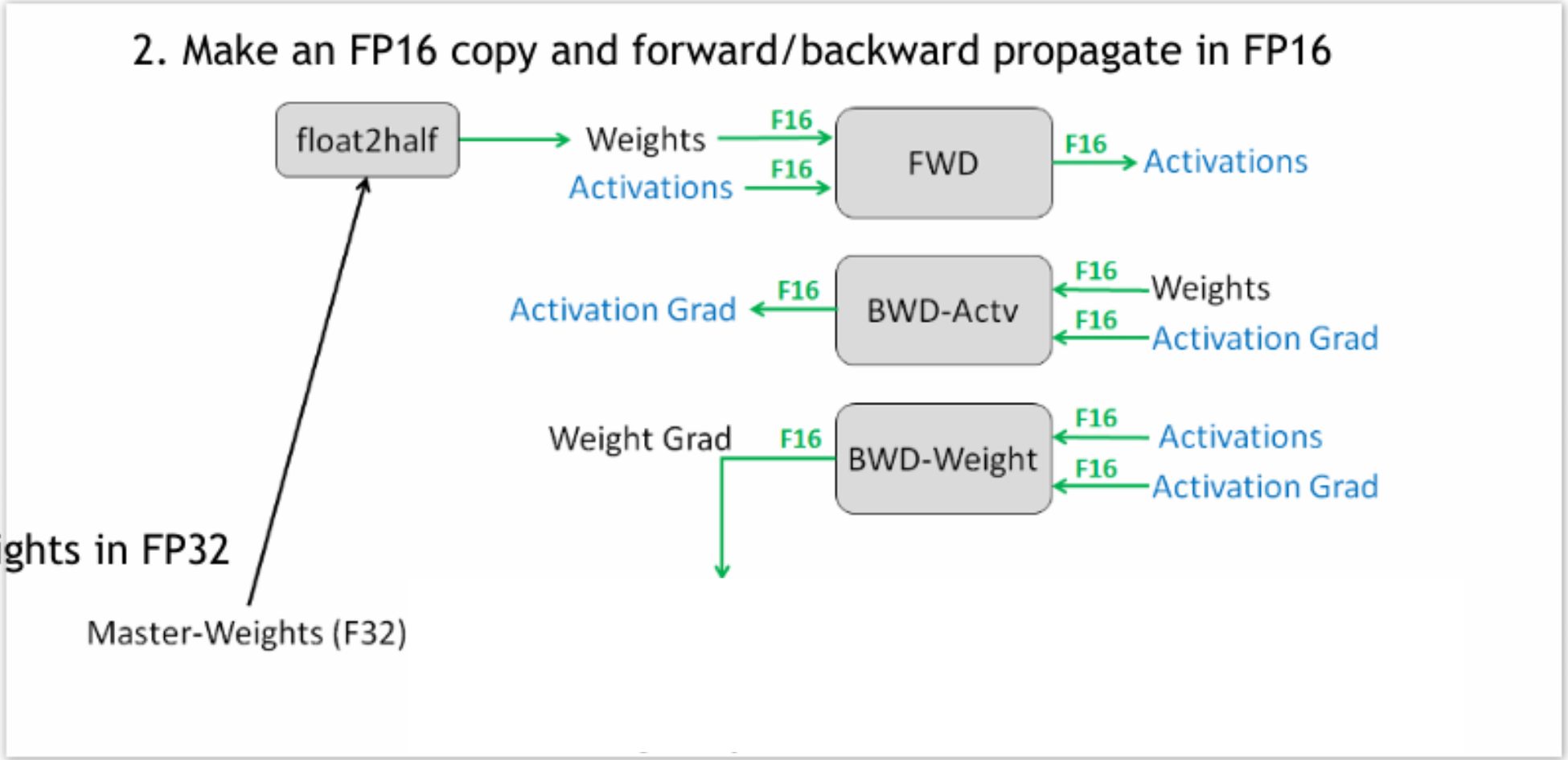
Automatic Loss Scaling: Frees Users from Choosing a Scaling Factor

- 1. Start with very large scale factor (e.g., 2^24)
- 2. If gradients overflows (with Inf or a Nan): decrease the scale by 2 and skip the update
- 3. If no overflows have occurred for some time: increase the scale by 2 (e.g., 2000 iterations)

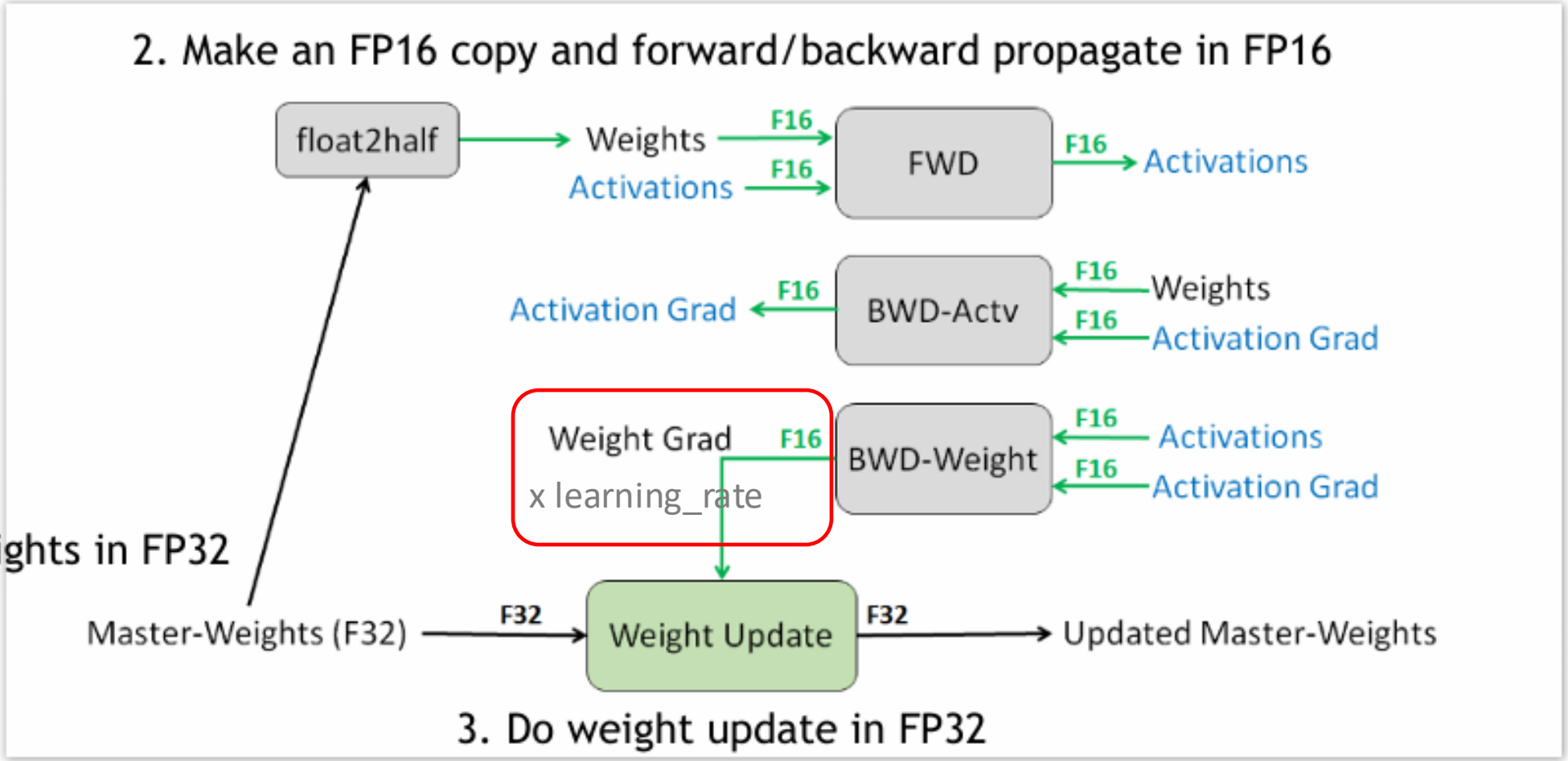


[Train With Mixed Precision - NVIDIA Docs]

Master Weights: Illustration

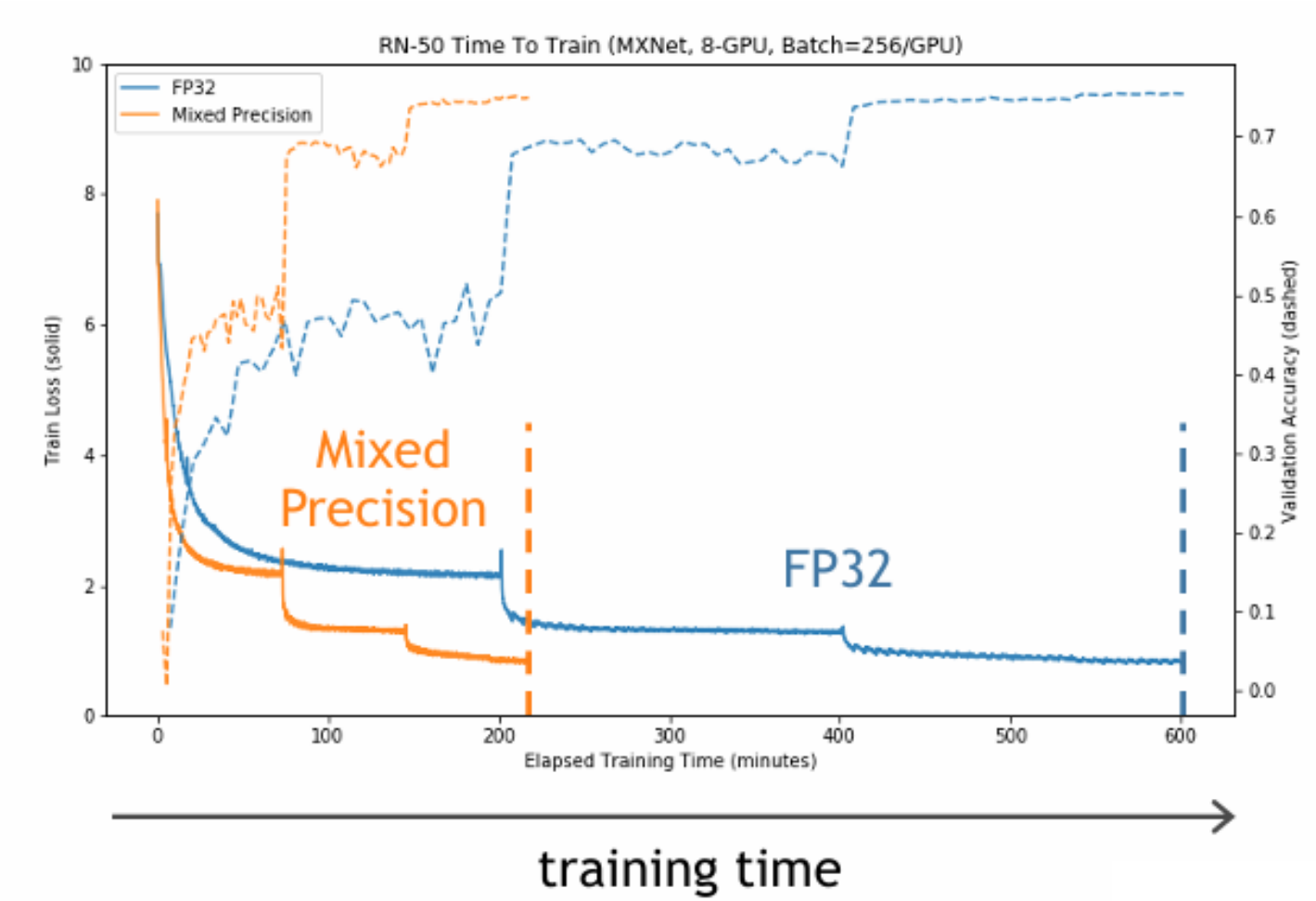


Master Weights: Illustration



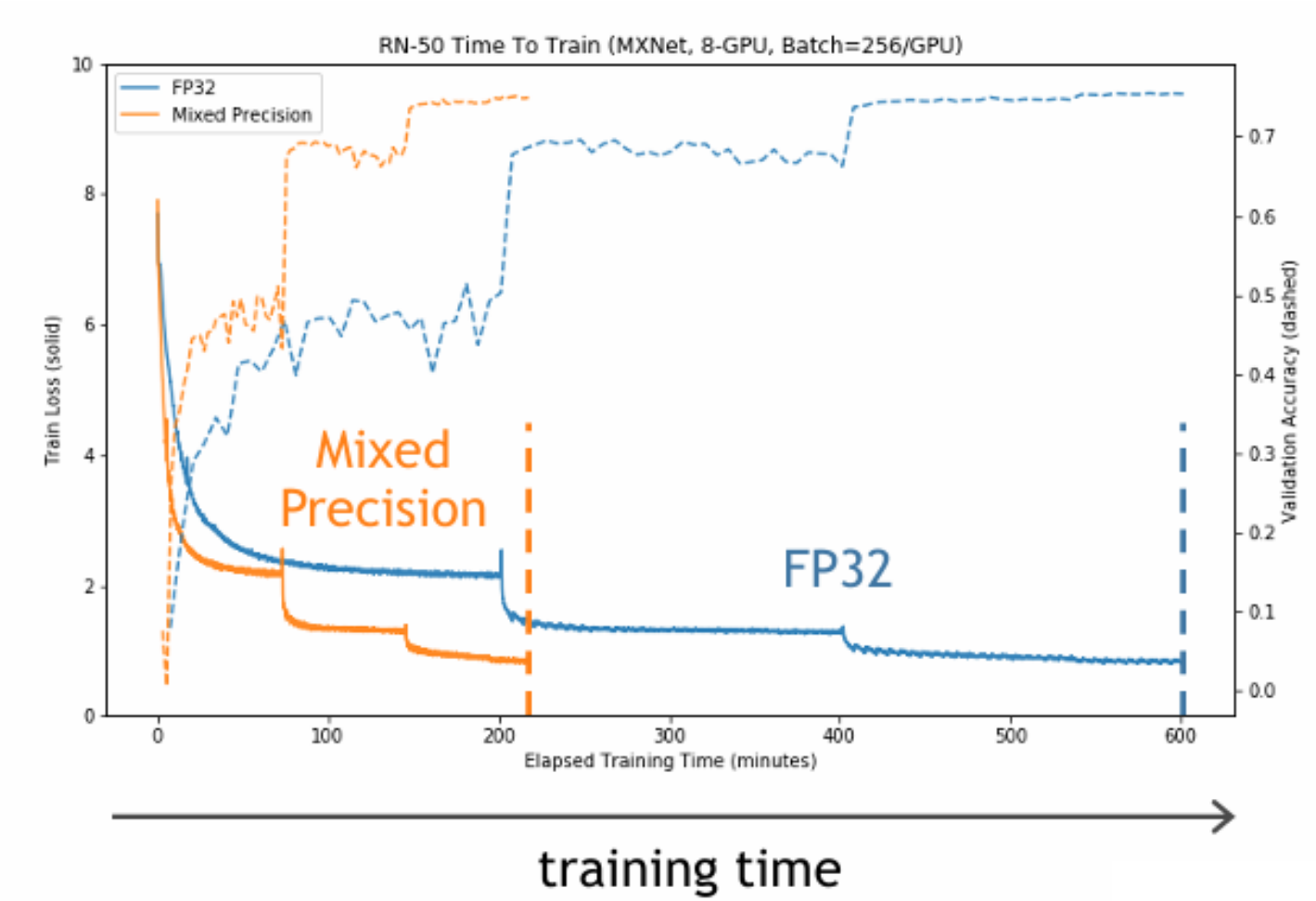
Mixed Precision Training Example

- ResNet-50 training for ImageNet classification
  - 8 GPUs on DGX-1



Mixed Precision Training Example

- ResNet-50 training for ImageNet classification
  - 8 GPUs on DGX-1
- Comparing to FP32 training
  - **~3x speedup**
  - **Equal accuracy**
  - No hyperparameters changed



Mixed Precision is Generalizable

- Works across a wide range of tasks, problem domains, deep neural network architectures

Image Classification
AlexNet
DenseNet
Inception
MobileNet
NASNet
ResNet
ResNeXt
ShuffleNet
SqueezeNet
VGG
Xception

Detection / Segmentation
DeepLab
Faster R-CNN
Mask R-CNN
SSD
NVIDIA Automotive
RetinaNet
UNET

Recommendation
DeepRecommender
NCF

Generative Models (Images)
DLSS
GauGAN
Partial Image Inpainting
Progress GAN
Pix2Pix

Speech
Deep Speech 2
Jasper
Tacotron
Wave2vec
WaveNet
WaveGlow

Language Modeling
BERT
BigLSTM
Gated Convolutions
mLSTM
RoBERTa
Transformer XL

Translation
Convolutional Seq2Seq
Dynamic Convolutions
GNMT (RNN)
Levenshtein Transformer
Transformer (Self-Attention)

Mixed Precision Trains Faster: Drastically Reduces Training Time

Task	Model	Speedup
Image Classification	ResNet-50	3.6x
	DenseNet 201	2.2x*
	Xception	2.1x*
Detection / Segmentation	SSD	2.5x**
	Mask R-CNN	1.5x
	RetinaNet	2.0x

* 2x batch size ** Larger batch size

Task	Model	Speedup
Translation	GNMT	2.3x
	Transformer	2.9x 4.9x**
	Convolutional Seq2Seq	2.5x*
Speech	Deep Speech 2	4.5x**
	Wav2letter	3.0x*
	WaveGlow	1.9x
Language Modeling	mLSTM	4.0x**
	BERT	3.3x

Weeks to days

Days to hours

Hours to minutes

Mixed Precision Software

Mixed Precision Software

- Available for major DL frameworks

  PyTorch|DeepSpeed|Megatron|…

- Works with all types of optimizers
  - SGD, Adam, etc

- Works with multiple models, optimizers, and losses

Automatic Mixed Precision (AMP) with PyTorch

To control the operations being casted

```
model, optimizer = amp.initialize(model, optimizer, opt_level="O1")
```

O0	O1	O2
FP32 Training Leave everything in FP32.	**Mixed Precision Training** FP16 whitelist and FP32 blacklist ops.	**FP16 Training** FP16 model/data with FP32 batchnorm.

Summary: Mixed Precision Training

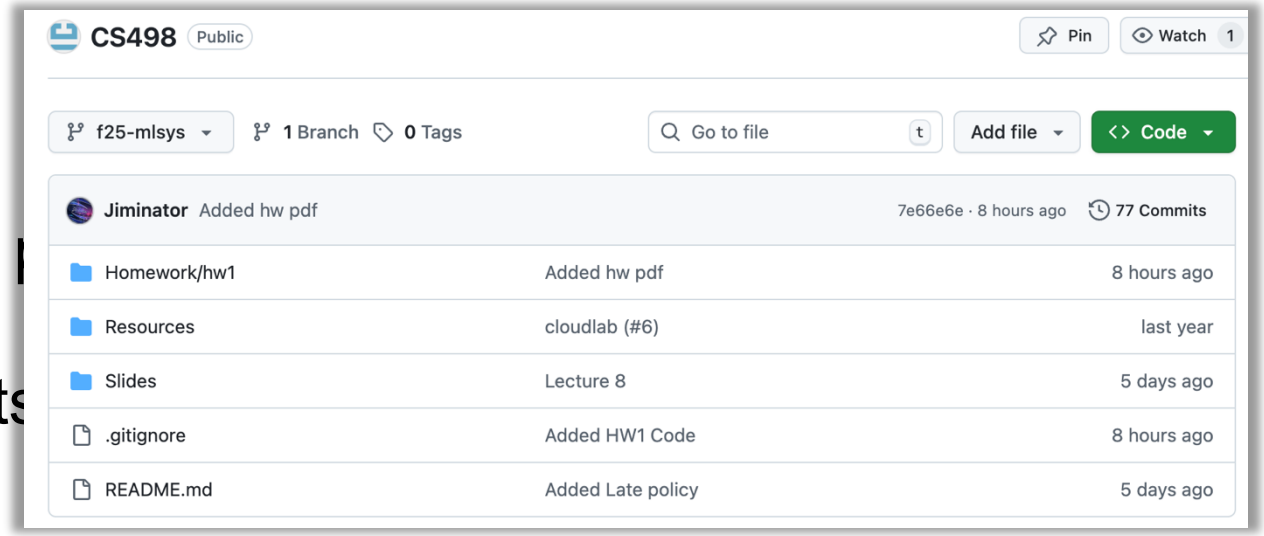
- **Mixed precision improves efficiency but may hurt quality**
  - FP16 vs. INT8
- **Automatic Scaling at runtime may help quality**
  - E.g., scaling the gradient to prevent underflow

Assignment 1 is Released (Due on 10/15)

Five short questions (10 pts)

Lab: Parameter-server based DP (10 p

Lab: All-reduce implementation (30 pts



NO Class this Friday! Meeting to discuss project ideas (SC 3128)

Project proposal due on 10/01!